

UNIT-IV: Machine Learning for Time Series Forecasting

Jupyter Notebook: [Link](#)

1. Supervised Learning Algorithms for Time Series

Time series forecasting can use **supervised learning** by converting time series into a supervised problem: predicting future value(s) based on past observations.

Step 1: Create Lag Features

Lag features are past values used to predict the future.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns

from matplotlib import pyplot as plt
import warnings
warnings.filterwarnings('ignore')

[2]: data = {'sales': [100, 120, 130, 150, 170, 200, 220]}
df = pd.DataFrame(data)

# Creating lag features
df['lag1'] = df['sales'].shift(1)
df['lag2'] = df['sales'].shift(2)
df.dropna(inplace=True)
print("Output\n")
print(df)
```

Output

	sales	lag1	lag2
2	130	120.0	100.0
3	150	130.0	120.0
4	170	150.0	130.0
5	200	170.0	150.0
6	220	200.0	170.0

2.1 Linear Regression for Time Series

Linear regression predicts a dependent variable y using independent variables X . For time series:

$$Y_t = \beta_0 + \beta_1 Y_{t-1} + \beta_2 Y_{t-2} + \dots + \epsilon$$

where,
 Y = Dependent variable
 X = independent variable

Code Example:

Linear Regression

```
[3]: from sklearn.linear_model import LinearRegression
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import mean_squared_error

      # Features and target
      X = df[['lag1', 'lag2']]
      y = df['sales']

      # Split data
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, shuffle=False)

      # Train Linear Regression
      model = LinearRegression()
      model.fit(X_train, y_train)

      # Predictions
      y_pred = model.predict(X_test)
      print("MSE:", mean_squared_error(y_test, y_pred))
```

MSE: 55.5555555555553

2.2 Decision Tree Regression

Decision Trees can model **non-linear relationships** in time series data.

Code Example:

Decision Tree Regressor

```
[5]: from sklearn.tree import DecisionTreeRegressor

tree_model = DecisionTreeRegressor(max_depth=3)
tree_model.fit(X_train, y_train)
y_tree_pred = tree_model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_tree_pred))

MSE: 1700.0
```

Advantages: Captures non-linear patterns, easy to interpret.

Disadvantages: Prone to overfitting, may need pruning.

2.3 Random Forest for Time Series

Random Forest is an **ensemble of Decision Trees**, reducing overfitting by averaging multiple trees.

Random Forest Regressor

```
[6]: from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_rf_pred = rf_model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_rf_pred))

MSE: 2384.8400000000001
```

Advantages: High accuracy, handles non-linear data.

Disadvantages: Slower for very large datasets, less interpretable.

2. Time Series Regression & Feature Engineering

Feature engineering is crucial for forecasting.

Common Features:

1. **Lag features** – Past values.
2. **Rolling statistics** – Mean, std over last *n* periods.
3. **Time-based features** – Day of week, month, holidays.

Example: Adding Rolling Mean

Adding a rolling mean

```
[7]: df['rolling_mean'] = df['sales'].rolling(window=2).mean().shift(1)
```

```
[8]: df.head()
```

```
[8]:
```

	sales	lag1	lag2	rolling_mean
2	130	120.0	100.0	NaN
3	150	130.0	120.0	NaN
4	170	150.0	130.0	140.0
5	200	170.0	150.0	160.0
6	220	200.0	170.0	185.0

3. Ensemble Methods for Time Series Forecasting

Ensemble methods combine multiple models to improve predictions.

Types of Ensemble Methods:

1. **Bagging (Bootstrap Aggregation)** – Random Forest.
2. **Boosting (Sequential)** – XGBoost, LightGBM.
3. **Stacking** – Combine predictions from multiple models.

Example: Using Random Forest + Linear Regression

Stacked Prediction

```
[10]: import numpy as np

# Stack predictions
stacked_pred = (y_pred + y_rf_pred) / 2
print("Stacked MSE:", mean_squared_error(y_test, stacked_pred))

Stacked MSE: 497.098888888888963
```

4. Model Selection for Time Series

- Use a **time-aware split** (no random shuffling).
- Evaluation metrics:
 - **MSE** (Mean Squared Error)
 - **RMSE** (Root Mean Squared Error)
 - **MAE** (Mean Absolute Error)
- Cross-validation: **TimeSeriesSplit** in scikit-learn.

Time Series Split

```
[11]: from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=3)
for train_index, test_index in tscv.split(X):
    print("Train:", train_index, "Test:", test_index)

Train: [0 1] Test: [2]
Train: [0 1 2] Test: [3]
Train: [0 1 2 3] Test: [4]
```

Conclusion

- Time series forecasting can be treated as supervised learning using **lag features**.
- **Linear Regression**: Simple, interpretable.
- **Decision Tree / Random Forest**: Handles non-linear patterns.
- **Feature Engineering**: Lag features, rolling stats, time-based features improve predictions.
- **Ensemble Methods**: Improve accuracy and reduce overfitting.
- **Model Selection**: Always respect temporal order. Use metrics like MSE, MAE.

Flow Diagram of Forecasting Pipeline:

